Tema 5: Sistemas de recuperación de la información, RAG y Langchain

EBIS Business Techschool

Contenidos

- Fundamentos matemáticos de la recuperación de información
- Representación de la información I: Sparse vs Dense
- Representación de la información II: Métodos modernos de retrieval
- Indexación y búsqueda eficiente
- Sistemas de almacenamiento y recuperación
- Construcción y evaluación de sistemas de recuperación
- Retrieval Augmented Generation

www.ebiseducation.com

Agenda de la sesión

EBIS Business Techschool

De la búsqueda exacta al problema de escala

Retomaremos el concepto de vecino más cercano y veremos qué ocurre cuando la colección pasa de unos pocos vectores a millones. Analizaremos el coste de comparar una query contra todo el corpus y por qué la búsqueda exacta deja de ser viable en muchos escenarios.

HNSW: búsqueda mediante grafos de proximidad

Estudiaremos la intuición detrás de Hierarchical Navigable Small World, cómo organiza los vectores mediante conexiones entre vecinos y cómo utiliza una estructura jerárquica para navegar rápidamente hacia regiones relevantes del espacio.

Trade-offs y elección del índice

Cerraremos comparando búsqueda exacta, HNSW e IVF en términos de **recall, latencia, memoria, coste de construcción y actualización**, entendiendo que no existe un índice universalmente mejor y que su configuración depende del problema.

Approximate Nearest Neighbors (ANN)

Introduciremos la idea de búsqueda aproximada y el principal compromiso que aparece en estos sistemas: sacrificar una pequeña parte del recall para reducir drásticamente la latencia y el coste computacional.

IVF y particionado del espacio vectorial

Veremos un enfoque diferente basado en dividir el espacio en regiones o clusters y buscar únicamente dentro de las zonas más prometedoras. Introduciremos también, el papel de técnicas como Product Quantization cuando necesitamos reducir memoria y acelerar todavía más la búsqueda.

Recap & Q&A

Resumen de conceptos clave y resolución de dudas.

De la búsqueda exacta al problema de escala

EBIS Business Techschool

Buscar el vecino más cercano

EBIS Business Techschool

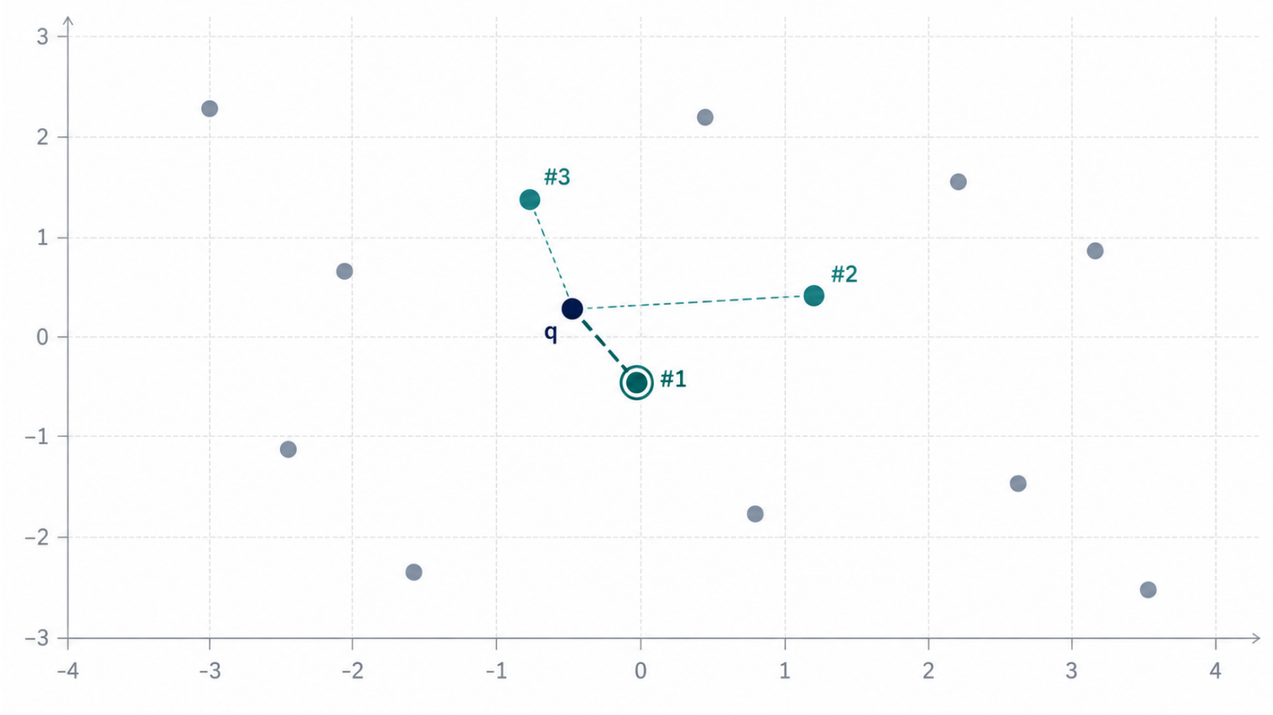
Dada una consulta representada por un vector **q**, queremos encontrar dentro de una colección los elementos cuya representación sea más próxima según una determinada función de distancia o similitud.

Si usamos una distancia:

$$NN(q) = \arg\min_{x \in X} d(q, x)$$

Si usamos una medida de similitud:

$$NN(q) = \arg\max_{x \in X} sim(q, x)$$



En retrieval normalmente no buscamos un único vecino, sino los **k elementos mejor posicionados:**

Top-k Nearest Neighbors

Hasta ahora hemos asumido que podemos comparar la query con **todos los vectores de la colección**.

Exact Search: comparar contra todo

La estrategia más directa consiste en calcular la distancia o similitud entre la query y **cada vector de la colección**, ordenar los resultados y seleccionar los mejores.

$$q \rightarrow \{x_1, x_2, \ldots, x_N\} \rightarrow \{s_1, s_2, \ldots, s_N\} \rightarrow Top - k$$

Exactitud

Garantiza encontrar los verdaderos vecinos más cercanos se la métrica utilizada.

Simplicidad

No necesitamos construir una estructura de búsqueda adicional.

Coste

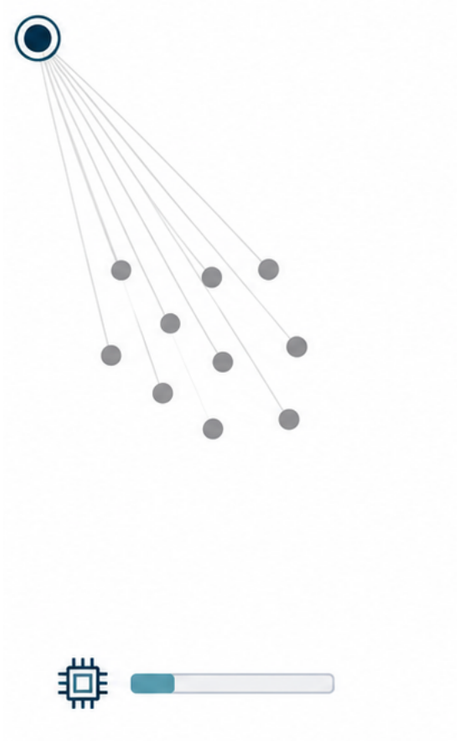
Cada consulta requiere evaluar **todos los candidatos**.

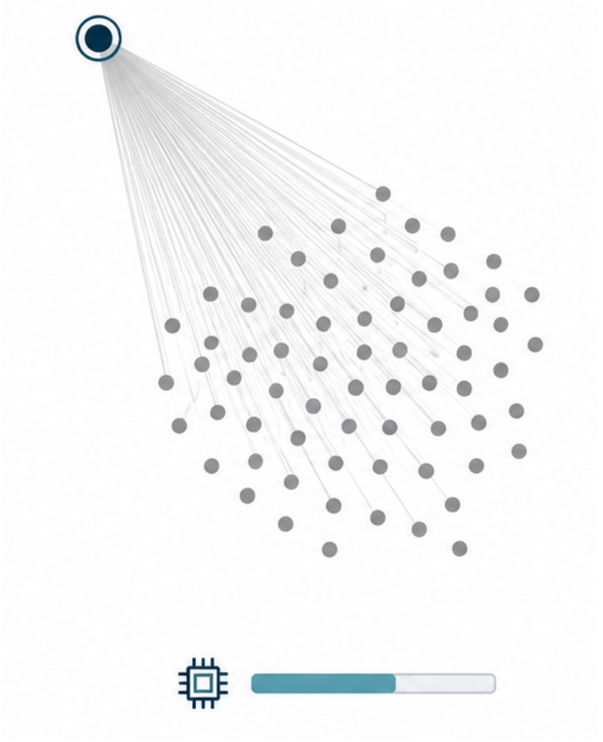
Para una colección de ***N* vectores de dimension *d***, una búsqueda exhaustiva requiere aproximadamente:

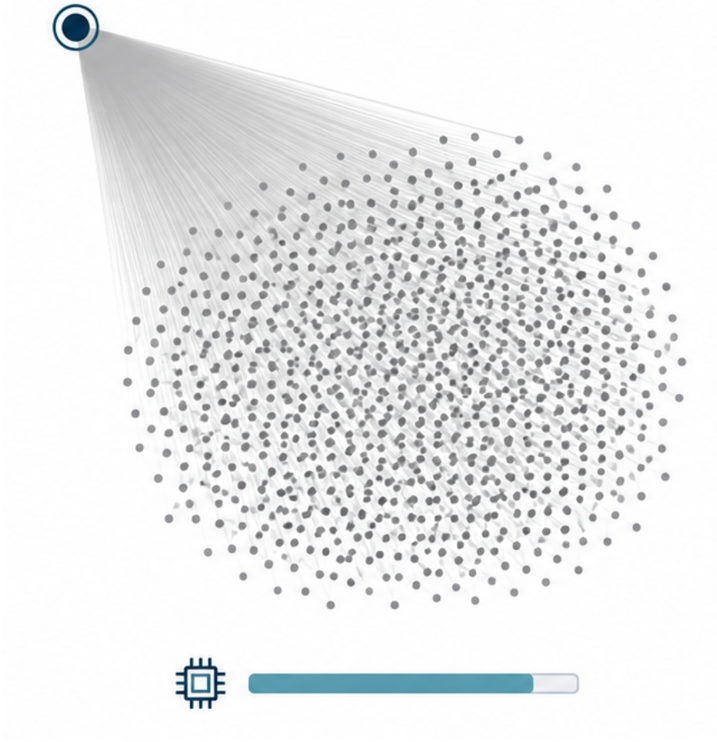
$$O(Nd)$$

El problema no es la fórmula: es cuántas veces la ejecutamos

Calcular una similitud entre dos vectores es barato. El problema aparece cuando debemos repetir esa operación **millones o miles de millones de veces por consulta**.



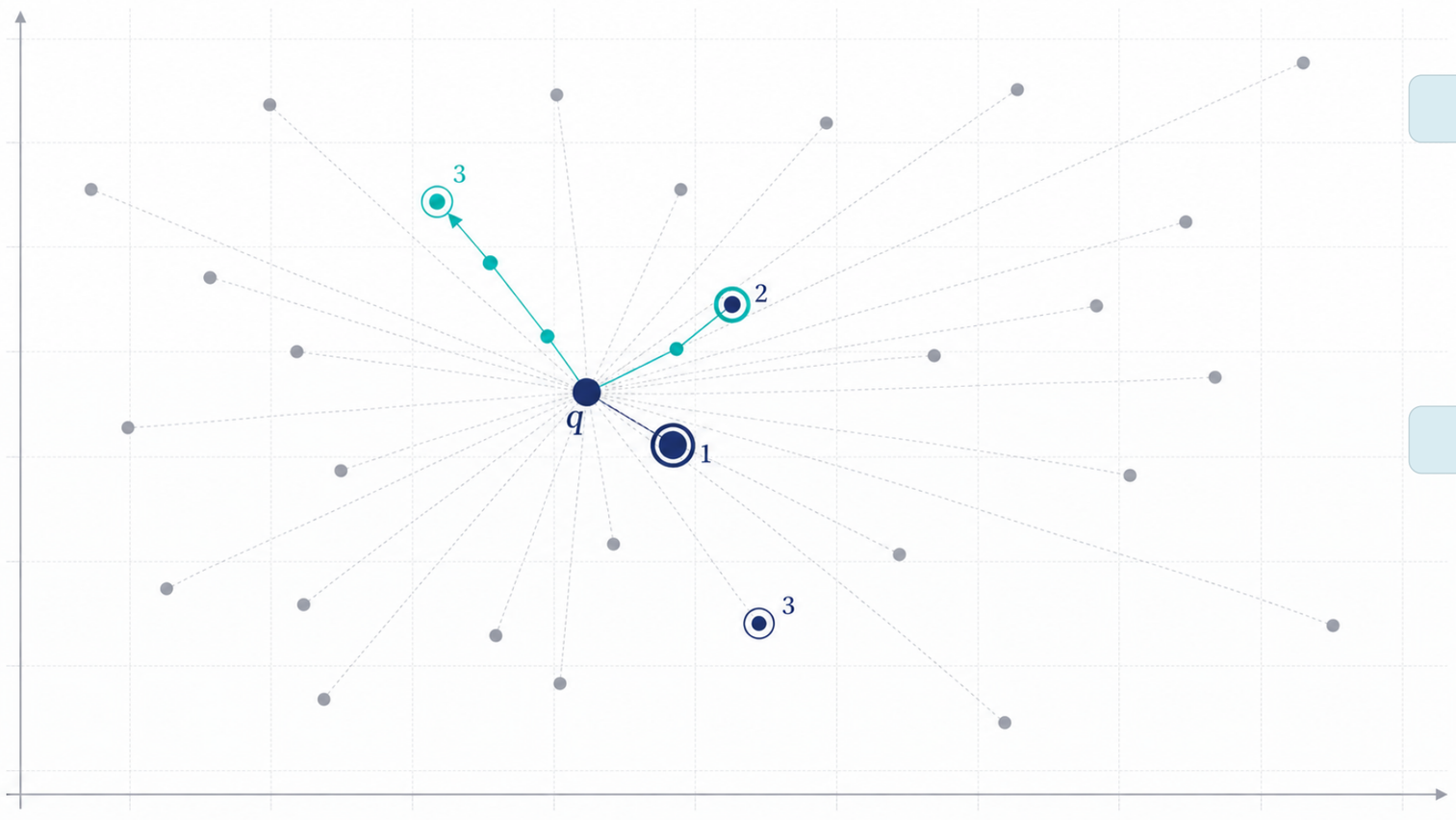




	1.000 vectores	1 millón vectores	1.000 millones vectores
768 dimensiones	768.000 componentes	768 millones	768.000 millones
1.536 dimensiones	1,536 millones	1.536 millones	1,536 billones

¿Necesitamos realmente encontrar el vecino exacto?

En muchos sistemas de recuperación, encontrar siempre el vecino matemáticamente óptimo puede ser menos importante que responder **mucho más rápido con resultados prácticamente equivalentes**.



Exact Search

Examina toda la colección.

- **Máxima exactitud.**
- **Mayor coste de consulta.**

Approximate Search

Explora únicamente una parte prometedora del espacio.

- **Respuesta mucho más rápida.**
- **Posibilidad de perder algún vecino real.**

Encontrar vecinos suficientemente buenos utilizando una fracción del coste.

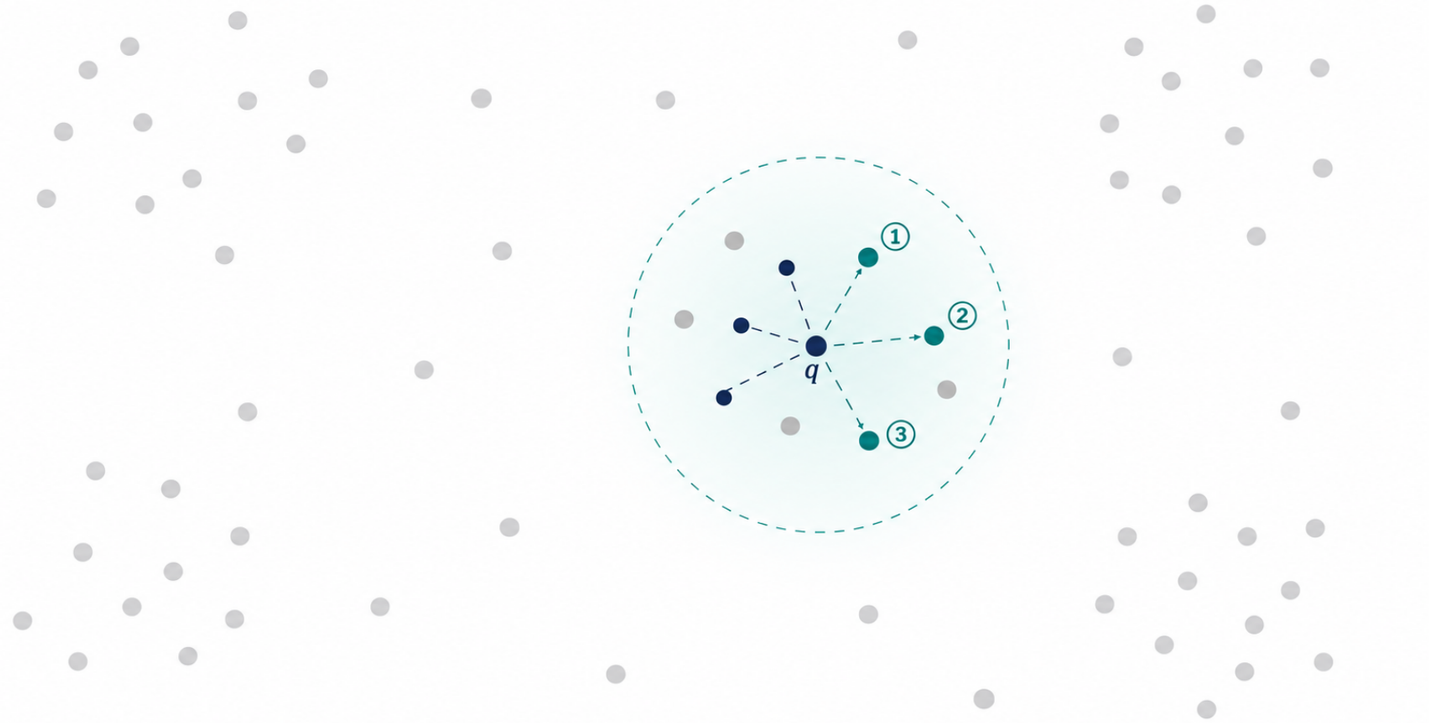
Approximate Nearest Neighbors

EBIS Business Techschool

Approximate Nearest Neighbors: buscar sin recorrer todo el espacio

EBIS Business Techschool

Un algoritmo ANN construye o utiliza una estructura que permite descartar grandes regiones de la colección y concentrar la búsqueda en los candidatos con mayor probabilidad de encontrarse cerca de la query.



Exact Search

N candidatos → evaluar N

ANN Search

N candidatos → explorar solo una fracción

El resultado puede no contener siempre los vecinos matemáticamente exactos, pero pretende recuperar prácticamente los mismos resultados con mucho menos trabajo.

ANN no aproxima la similitud: aproxima la búsqueda de los vecinos.

Puedes seguir calculando cosine similarity o L2 exactamente entre los candidatos finalmente inspeccionados; lo que evitamos es comparar contra todos.

Aproximado no significa aleatorio

Los algoritmos ANN no seleccionan candidatos al azar. Utilizan una estructura construida a partir de la geometría de los datos para decidir qué partes del espacio merece la pena explorar.

1 **Indexar**

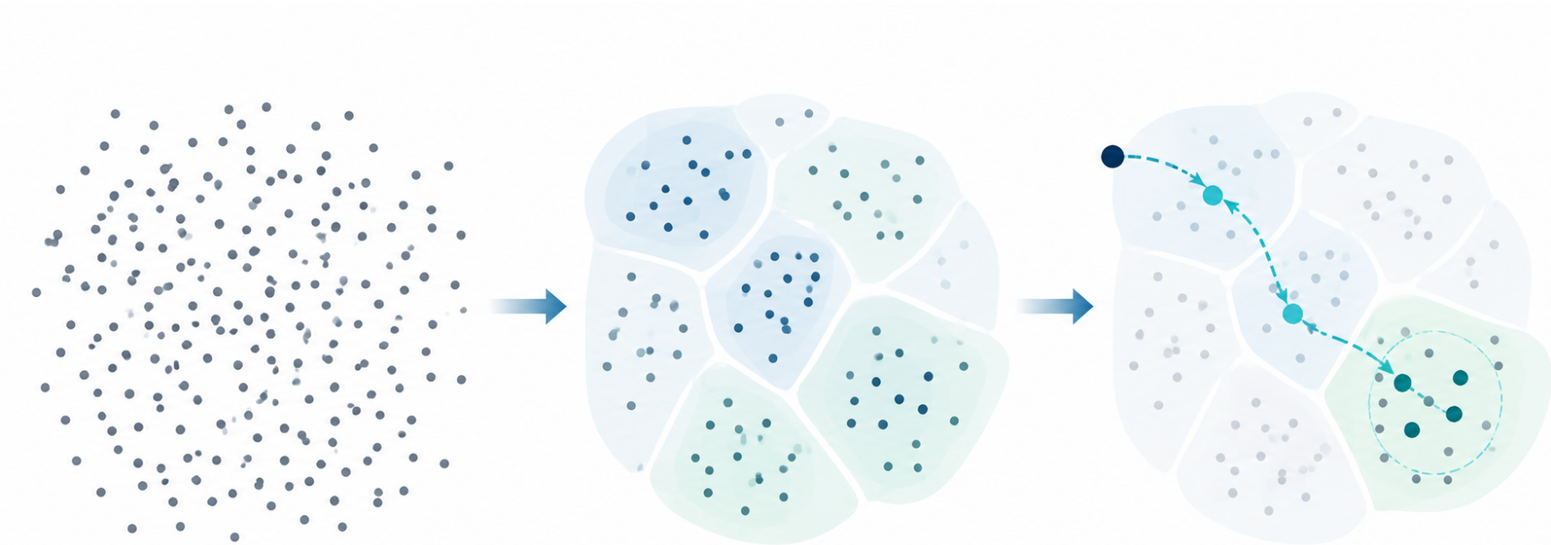
Analizar previamente la colección y construir una estructura auxiliar.

2 **Navegar**

Utilizar esa estructura para dirigir la query hacia regiones prometedoras.

3 **Verificar**

Comparar la query con un conjunto reducido de candidatos y recuperar los mejores.



Desplazamos parte del coste desde el momento de la consulta hacia una fase previa de construcción del índice.

Más trabajo offline → menos trabajo por query

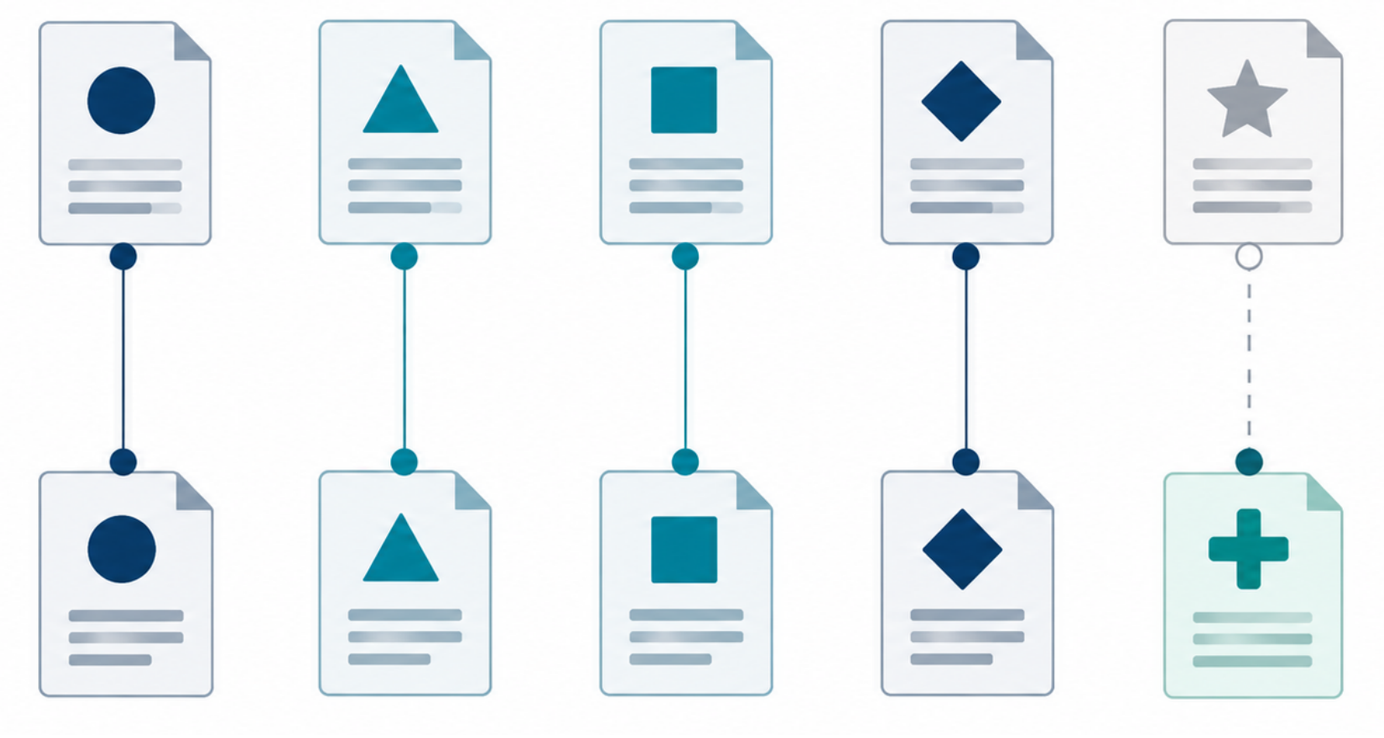
¿Cuánto perdemos al aproximar la búsqueda?

Podemos comparar los resultados ANN con los vecinos obtenidos mediante búsqueda exacta para medir qué proporción de los resultados correctos conseguimos recuperar.

$$Recall@k = \frac{|TopK_{ANN} \cap TopK_{Exact}|}{k}$$

Exact Top-5	A	B	C	D	E
ANN Top-5	A	B	C	E	F

$$Recall@5 = 4 / 5 = 0.8$$



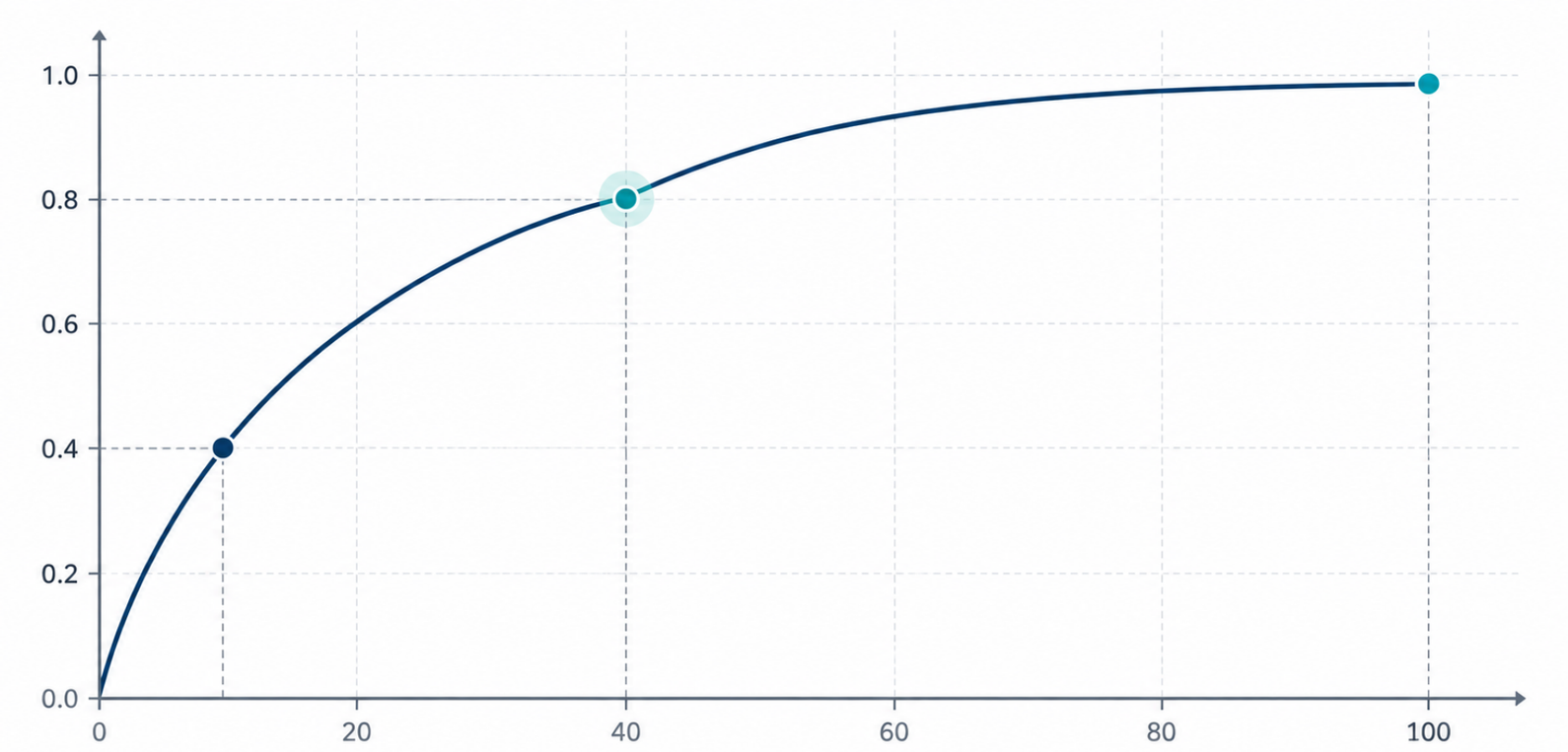
$Recall@k = 1$ → recuperamos todos los vecinos exactos del Top-k

$Recall@k < 1$ → alguno de los vecinos exactos se ha perdido

ANN permite intercambiar una pequeña pérdida de recall por una reducción potencialmente enorme de latencia.

El trade-off fundamental: recall vs latencia

Los índices ANN suelen permitir controlar cuánto esfuerzo dedicamos a cada búsqueda.



Más exploración

↑ candidatos evaluados ↑ latencia ↑ Recall@k

Menos exploración

↓ candidatos evaluados ↓ latencia ↓ Recall@k

No buscamos necesariamente maximizar Recall@k a cualquier precio. El objetivo práctico es encontrar un punto de operación que ofrezca suficiente calidad dentro de los requisitos de latencia y coste del sistema.

ANN es un problema de compromiso, no simplemente de velocidad.

HNSW: búsqueda mediante grafos de proximidad

EBIS Business Techschool

HNSW: Buscar navegando entre vecinos

HNSW organiza los vectores como un grafo de proximidad: cada elemento se conecta con otros vectores cercanos dentro del espacio.

En lugar de comparar la query contra toda la colección:

1. comenzamos desde un punto de entrada
2. nos movemos hacia vecinos cada vez más prometedores
3. detenemos la exploración cuando ya no encontramos mejores candidatos

La búsqueda se transforma de escanear una colección a navegar por una red de vecinos.

Small World: Conexiones locales, saltos globales

Un grafo formado únicamente por conexiones muy locales puede requerir muchos pasos para atravesar el espacio. HNSW aprovecha la idea de Small World Networks para reducir ese recorrido.

Conexiones cortas

Permiten explorar con precisión una región concreta.

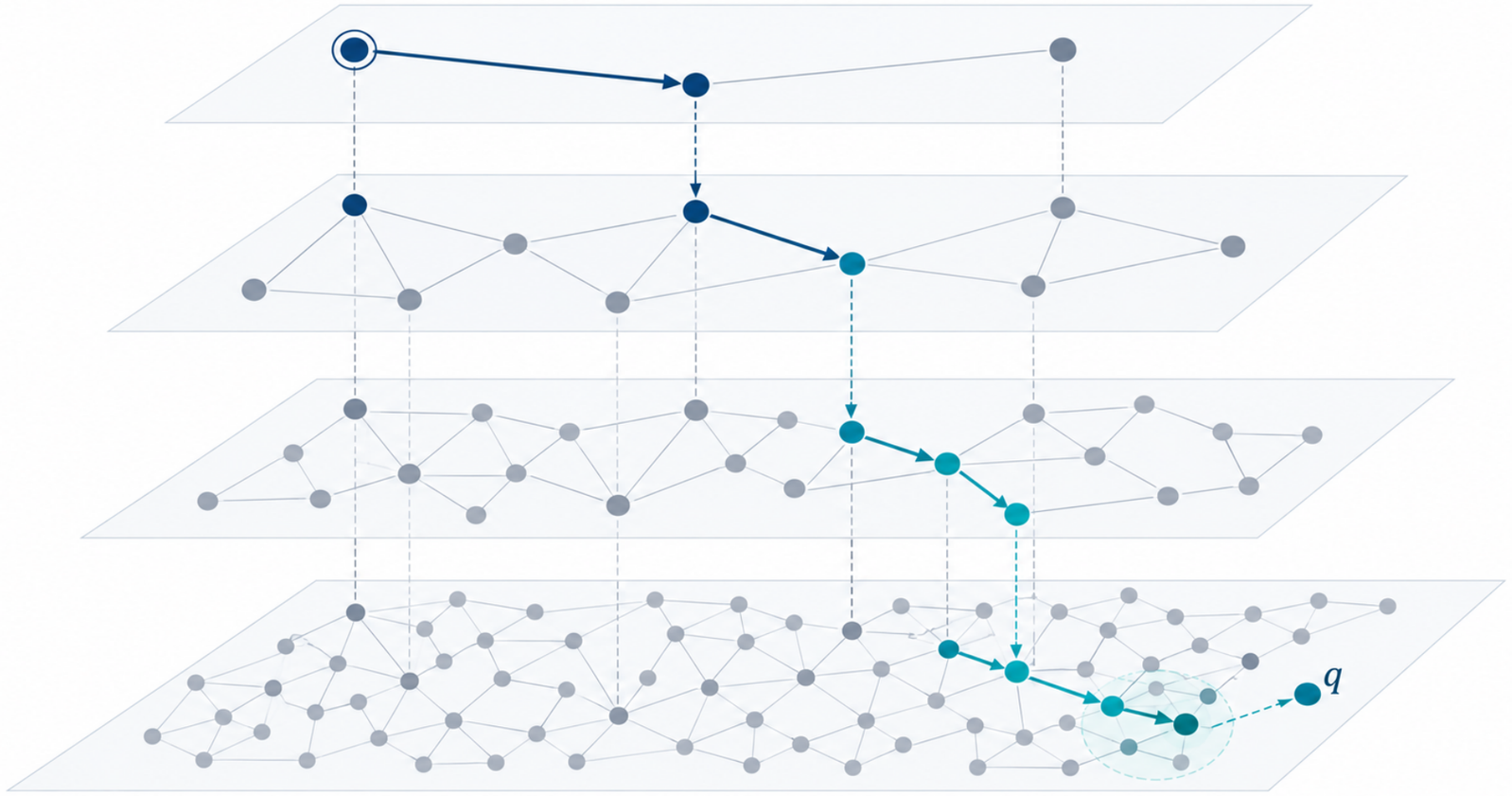
Conexiones largas

Permiten realizar grandes saltos entre regiones alejadas del espacio.

La combinación de ambos tipos de conexiones permite alcanzar rápidamente una zona prometedora y después refinar la búsqueda localmente.

La jerarquía de HNSW

HNSW organiza el grafo en múltiples capas. Las superiores contienen menos nodos y permiten desplazamientos grandes; las inferiores contienen progresivamente más detalle.



Capas superiores

Pocos nodos Navegación rápida entre regiones alejadas

Capas intermedias

Mayor densidad Refinamiento progresivo de la búsqueda

Capa base

Contiene todos los elementos Búsqueda precisa entre vecinos cercanos

Navegar rápido arriba → refinar progresivamente → buscar localmente abajo

La query desciende por la jerarquía utilizando en cada nivel el mejor candidato encontrado como punto de partida para el siguiente.

HNSW no aparece gratis: hay que construir el grafo

Cada nuevo vector debe incorporarse al índice buscando una región apropiada y estableciendo conexiones con otros nodos.

El proceso conceptual:

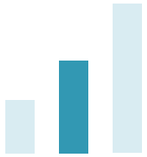
1. Buscar vecinos candidatos
2. Seleccionar conexiones útiles
3. Insertar el nodo en las capas correspondientes

El índice mantiene información adicional sobre las conexiones del grafo, por lo que consume más memoria que almacenar únicamente los vectores.

HNSW intercambia memoria y coste de construcción por búsquedas extremadamente rápidas.

Los parámetros que controlan HNSW

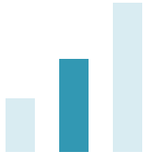
EBIS Business Techschool



M

Número aproximado de conexiones que mantiene cada nodo.

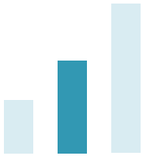
↑ *M* → más conectividad y memoria, normalmente mayor recall



ef_construction

Cantidad de candidatos explorados durante la construcción del índice.

↑ *ef_construction* → mejor índice, pero construcción más costosa



ef_search

Cantidad de candidatos explorados durante cada búsqueda.

↑ *ef_search* → mayor Recall@k, pero mayor latencia

INDEX BUILD *M, ef_construction*

QUERY TIME

***M* y *ef_construction* condicionan el índice. *ef_search* permite ajustar el trade-off en tiempo de consulta.**

HNSW no tiene una única configuración óptima: debemos encontrar el equilibrio entre recall, latencia, memoria y coste de construcción.

HNSW: búsqueda mediante grafos de proximidad

EBIS Business Techschool

IVF: Dividir el espacio antes de buscar

En lugar de navegar un grafo, IVF organiza los vectores en regiones del espacio asociadas a centroides.

1 **Construir centroides**

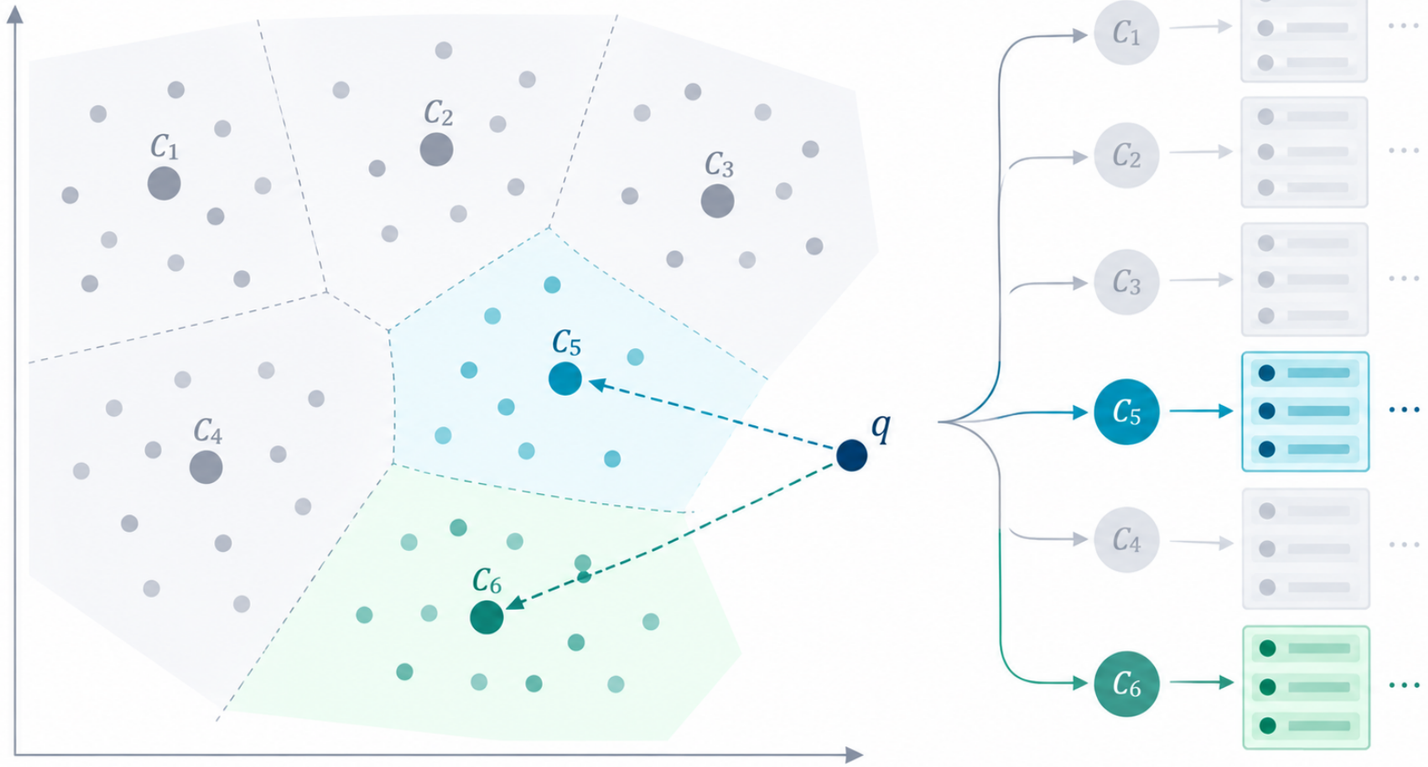
2 **Asignar cada vector a su región**

3 **Buscar solo en las regiones más prometedoras**

La búsqueda deja de recorrer toda la colección y pasa a explorar únicamente una fracción del espacio.

Cada región mantiene su propia lista de candidatos

EBIS Business Techschool



Una vez construido el particionado, cada vector se asigna a un centroide y queda almacenado dentro de su lista invertida.

$C_1 \rightarrow \{x_2, x_7, x_{14}, ...\}$

$C_2 \rightarrow \{x_1, x_9, x_{21}, ...\}$

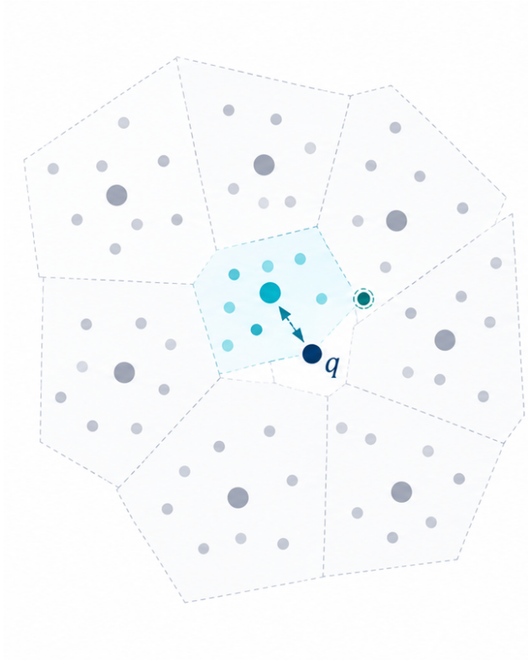
$C_3 \rightarrow \{x_3, x_4, x_{18}, ...\}$

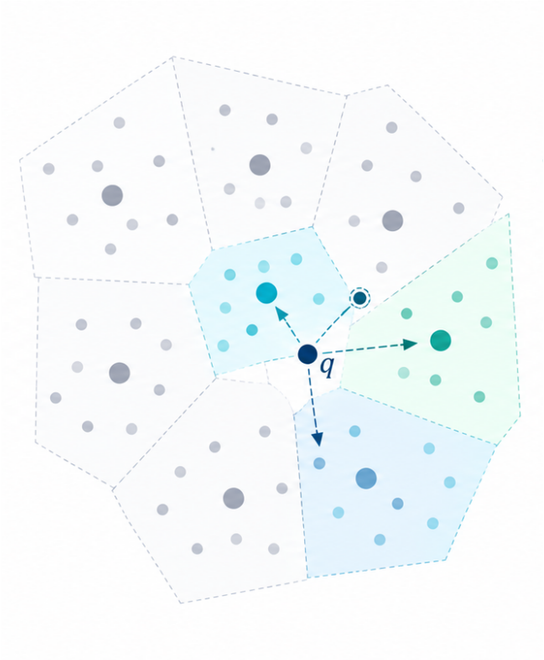
El ahorro depende de cuántas listas decidamos inspeccionar.

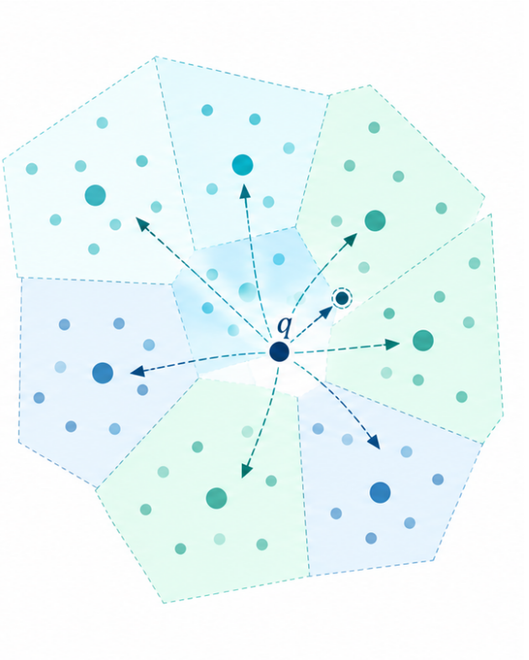
ANTE UNA QUERY 1 buscamos los centroides más cercanos → 2 accedemos a sus listas → 3 comparamos únicamente contra esos vectores

nprobe: controlar cuántas regiones exploramos

En IVF, la query no tiene por qué limitarse a una única región. Podemos explorar varios centroides cercanos para aumentar la probabilidad de recuperar los verdaderos vecinos.







nprobe = 1

- Muy pocas comparaciones
- Menor latencia
- Mayor riesgo de perder vecinos

nprobe intermedio

- Más candidatos
- Mejor Recall@k
- Coste moderado

nprobe alto

- Explora muchas regiones
- Recall cercano a exact search
- Mayor latencia

***nprobe* controla directamente el trade-off entre Recall@k y coste de búsqueda.**

Product Quantization: intercambiar precisión por memoria

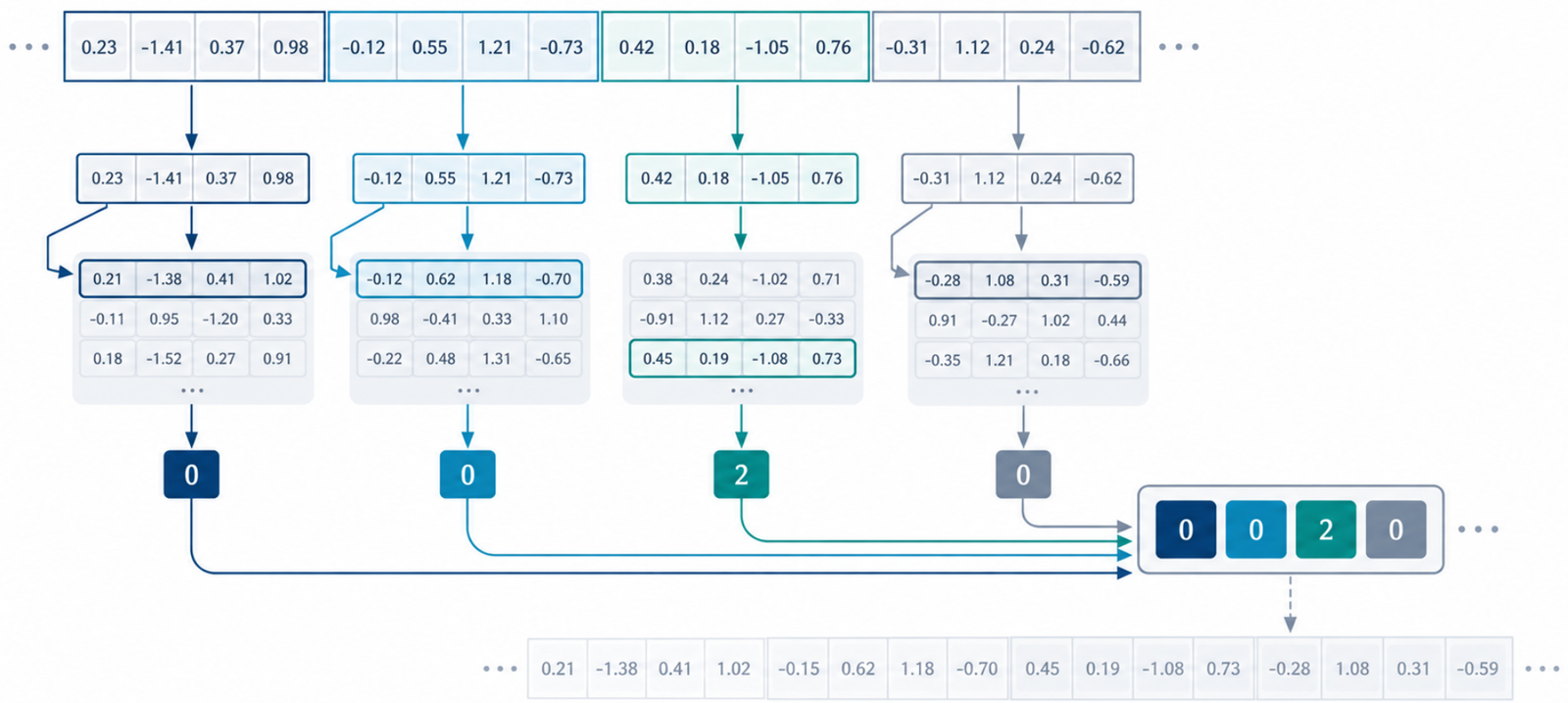
En colecciones muy grandes, el coste no está solo en buscar: también está en almacenar los vectores.

Product Quantization (PQ) aproxima cada vector mediante una representación compacta formada por códigos discretos.

Vector original
muchos valores en coma flotante

↓

Vector cuantizado
pocos códigos compactos



Menos memoria
Podemos almacenar colecciones mucho mayores.

Comparaciones más rápidas
Parte del cálculo puede realizarse sobre representaciones comprimidas.

Pérdida de precisión
La representación deja de ser exacta.

IVF reduce dónde buscamos. PQ reduce cuánto ocupa y cuánto cuesta comparar cada vector.

Ambos mecanismos pueden combinarse: **IVF-PQ**

Trade-offs y elección del índice

EBIS Business Techschool

No existe un índice universalmente mejor

Elegir un índice vectorial implica optimizar varias propiedades que compiten entre sí.

Recall

¿Cuántos de los verdaderos vecinos recuperamos?

Latencia

¿Cuánto tarda una consulta?

Memoria

¿Cuánto cuesta mantener la estructura y los vectores?

Construcción y actualización

¿Cuánto cuesta crear y mantener el índice?

Mejorar una dimensión suele implicar empeorar alguna otra.

La elección correcta depende de las restricciones del sistema, no solo del algoritmo.

Exact Search, HNSW e IVF: tres estrategias de búsqueda

	Exact Search	HNSW	IVF
Estrategia	Examinar todos los vectores	Navegar un grafo	Explorar regiones
Recall	Exacto	Muy alto configurable	Configurable
Latencia a gran escala	Alta	Muy baja	Baja
Memoria adicional	Baja	Alta	Moderada
Construcción del índice	Mínima	Costosa	Requiere clustering
Actualizaciones	Simples	Posibles, con coste	Dependientes de la implementación

Los valores son relativos y dependen del tamaño del corpus, dimensionalidad, hardware y configuración.

La estructura que acelera la búsqueda también introduce nuevos costes.

Elegir el índice a partir del problema

En lugar de empezar preguntando: ~~*"¿Cuál es el mejor índice?"*~~

Empezamos preguntando:

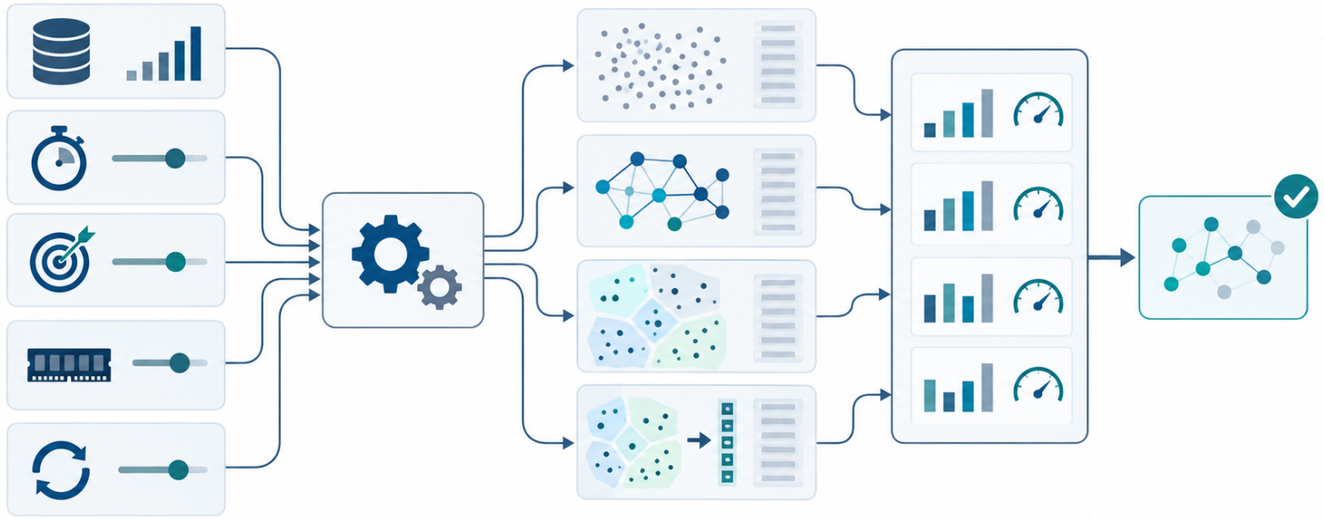
¿Cuántos vectores tenemos?
Miles no plantean el mismo problema que cientos de millones.

¿Qué latencia necesitamos?
Procesamiento offline y búsqueda interactiva tienen requisitos distintos.

¿Cuánto recall podemos sacrificar?
Algunas aplicaciones toleran aproximación; otras necesitan resultados casi exactos.

¿Cuánta memoria tenemos disponible?
HNSW puede comprar velocidad utilizando más memoria; PQ puede reducir drásticamente el almacenamiento.

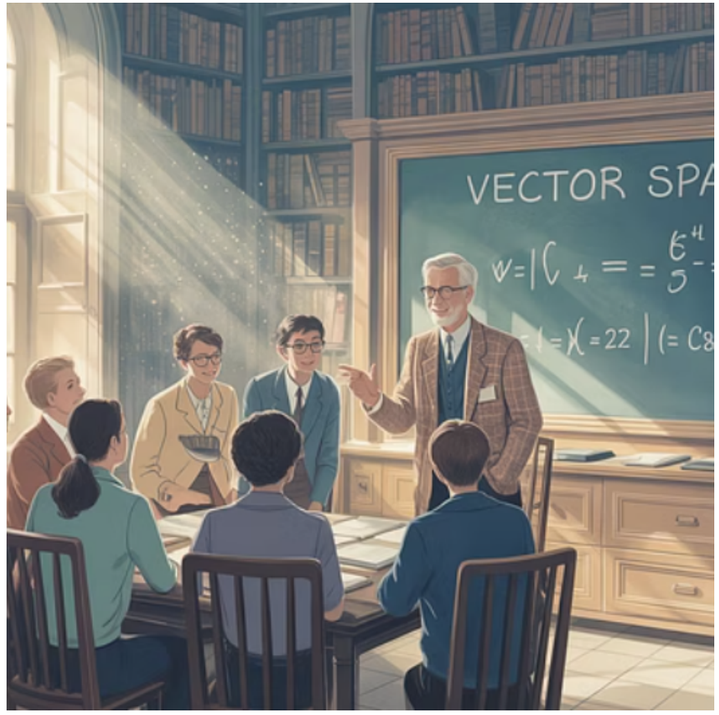
¿Cómo cambia la colección?
Índices estáticos y colecciones con inserciones frecuentes tienen necesidades diferentes.



Dataset + hardware + SLA + calidad requerida → configuración del índice

La decisión debe validarse empíricamente mediante benchmarks sobre los propios datos.

¡Gracias y hasta la próxima!



No dudéis en contactarme si tenéis dudas adicionales o necesitáis aclaraciones sobre cualquier aspecto del curso. Podéis encontrarme a través de:

Correo electrónico

albertoragmat@gmail.com

LinkedIn

Alberto Ruiz-Arteaga González

Tutorías

Todos los viernes de **17:30 a 18:30 (hora de Madrid)** en el **Metacampus**.

EBIS Business Techschool

www.ebiseducation.com